CS261 – Software Engineering
Individual Report

Student ID: 5615699
Team Number: 36
Project Title: airport modelling project

31st March 2026

Contents

1 Requirements and Design **2**
1.1 Requirements . . . 2
1.2 System Design . . . 3

2 System Description **6**
2.1 Core Features . . . 6
2.2 User Interface . . . 6

3 System Evaluation **11**
3.1 Requirements Coverage . . . 11
3.2 Testing and Validation . . . 12
3.3 User Interface Evaluation . . . 12

4 Team and Methodology **14**
4.1 Methodology Choice . . . 14
4.2 Methodology in Practice . . . 16
4.3 Risk Management . . . 17

5 Individual Contribution **17**
5.1 Overview of Contributions . . . 17
5.2 Technical and Non-Technical Contributions . . . 17
5.3 Skills, Strengths, and Weaknesses . . . 18

6 Conclusion **18**

1 Requirements and Design

1.1 Requirements

The requirements document captured all core functional requirements from the client specification using the MoSCoW prioritisation scheme [1]. This allowed us to prioritise key functionality before considering extra features. If the requirements document were to be improved, the number of must-have requirements should have been decreased. The use of tables to represent the data over time (e.g. requirements 3.4 - 3.10) or the presentation of any extra data other than the key statistics (e.g., the pop-out menu) was not mentioned in the client specification, but all the requirements for those features were labelled as must-haves.

Several developer-facing requirements lacked measurable acceptance criteria, such as Requirement 9.1.d: "The simulation must run without any excessive memory usage, significant slowdown, or system instability." where the words "excessive", "significant" and "instability" are impossible to formally verify through testing and could have been replaced with "The simulation must use no more than 512MB of RAM during any runtime period, and must complete a 7-day simulated period with 5 runways within 60 seconds on a standard consumer laptop". From a software engineering perspective, useful non-functional requirements should follow the SMART criteria, meaning they are Specific, Measurable, Achievable, Relevant, and Time-bound [2], so future versions of the document should specify concrete thresholds such as maximum memory consumption or response time limits.

In general, the Requirements were adhered to, but some were changed or improved. Below is a table with each section of our requirements and any major changes or improvements we made to the requirements.

Table 1: Team Requirement Coverage.

Requirements Section	**Were they completed**	**Further Comments**
Input Parameters and Data Generation	No	Requirement 1.1 was extended to allow configurable inbound and outbound flow, adding greater flexibility, allowing the airport manager to test the maximum airport traffic. Requirement 1.4 was changed so that some fields had default values, making it easier to set up a new simulation to test specific parameters.
Simulation information	No	Requirement 2.3 was not met, and instead changed so that all rows in the pop-up menu act as buttons, which can display individual plane and runway data more clearly. This keeps all plane data in one place and lets the other tables function.
User interface	Yes	Requirement 3.1 was altered to include a compare simulations page, allowing the user to compare any simulations that happened in the past. Some table column names were altered for clarity. Holding pattern and Take-off queue size graphs were also added since they added a useful view on congestion, which the original requirements did not cover.
Arrival Modelling	Yes	Requirement 4.3 was changed because the clients clearly stated during the consultation that all emergencies are to be treated with the same priority. Hence, our code was also altered to a binary system of either emergency or non-emergency, with time of entry into the simulation deciding the priority in each group.
Departure Modelling	Yes	All requirements met, and no significant improvements were made.

Requirements Section	Were they completed	Further Comments
Runways	No	Requirement 6.4 was changed so that runway operational status changes are pre-scheduled before the simulation, instead of being applied dynamically. This also removed an inconsistency between 6.4.c and 6.4.d, which talked about live and pre-scheduled changes. Mid-simulation input could introduce synchronisation issues between SimLoop and the application thread. Given the five-week development period, this simplification was required to maintain a manageable scope.
Fuel Data	Yes	Requirement 7.1 has been improved using a formula that takes into account its speed, amount of fuel it can burn, as well as the normal distribution mentioned in the initial requirement, allowing for further improvement and scalability in the future if different plane models or arrival speeds are taken into account.
System Requirements - Testing	Yes	All requirements met, and no significant improvements were made.
Performance Requirements	Yes	All requirements met, and no significant improvements were made.
Reliability and Robustness Requirements	Yes	All requirements met, and no significant improvements were made.

1.2 System Design

The System's design was inspired by the Model-View-Controller (MVC) architectural pattern, separating the UI and simulation into distinct components. This was appropriate for the project since it allowed the Modelling and frontend team to develop their sections in parallel with minimal dependencies, which could have taken time from developing extra features during the strict 5-week development timespan. From a software engineering perspective, the MVC pattern shows a clear separation of concerns and simplifies the process of testing and maintenance. [3].

JavaFX was chosen because of its thread-safe concurrency API through `javafx.concurrent.Service` and `javafx.concurrent.Task`. This was essential for running `SimLoop` and the Application Thread in parallel to keep the UI responsive. `SimLoop` extends `Task<ModelOutput>`, allowing a completed `ModelOutput` object to be handed to the JFXAT in a single safe handoff via the Task success event. The status bar is updated without polling via `SimLoop` by calling `updateProgress(currentTime, runtime)` and is then handed to JFXAT using `Platform.runLater()`. Wrapping `Task` in a `Service` allows it to be run or restarted with new parameters, even though a `Task` can only be run once.

In general, the choices from the design document were adhered to, but some key changes were made, which are outlined in the table below:

Design change	Reasoning
Tick duration	The tick duration was reduced from two minutes to one. Because of a two-minute tick, landing and take-off, which were also two minutes, were treated as instantaneous, misrepresenting the runway blocking behaviour specified by the client. It also meant all measurements of time had to be even, making values such as the 5-minute standard deviation of the Box-Muller transformation impossible to represent. The two-minute tick time does double the number of iterations of SimLoop, but that was considered an acceptable trade-off for the gain in simulation accuracy.
Extra output metrics	The original `ModelOutput` in the design document did not account for all data needed by the UI. Since the Modelling and UI were developed in parallel, the interface had not been sufficiently planned out beforehand. For example, the data needed to display the original graphs was not included. This shows how important it is to properly define interfaces before parallel development begins [4].
Colour scheme	The colour scheme diverged from the palette in the design document, from mid-tone blues to a darker set of Navy blues. Teal was removed completely from the colour palette because blue and teal can clash in a data-dense UI a blue-based palette is easier to maintain during parallel development.
Real-time updates	The design Document stated that the application would allow the user to see the live state of the simulation as it ran. However, the results are only shown after it completes. This decision was made due to a lack of time in a short development timespan.

The disconnect between the design and requirements documents is further evidenced by the `AirCraft` class, which underwent the most significant changes during implementation. Several attributes in the final implementation, such as `operator`, `origin`, `destination`, `altitude`, `fuel`, and `assignedRunway`, were all absent from the design document, although they were explicitly listed as needed in the requirements document. This disconnect between the two documents should have been detected during a formal review process. Figures 1 and 2 show the original planned class diagram alongside the final implemented class diagram:

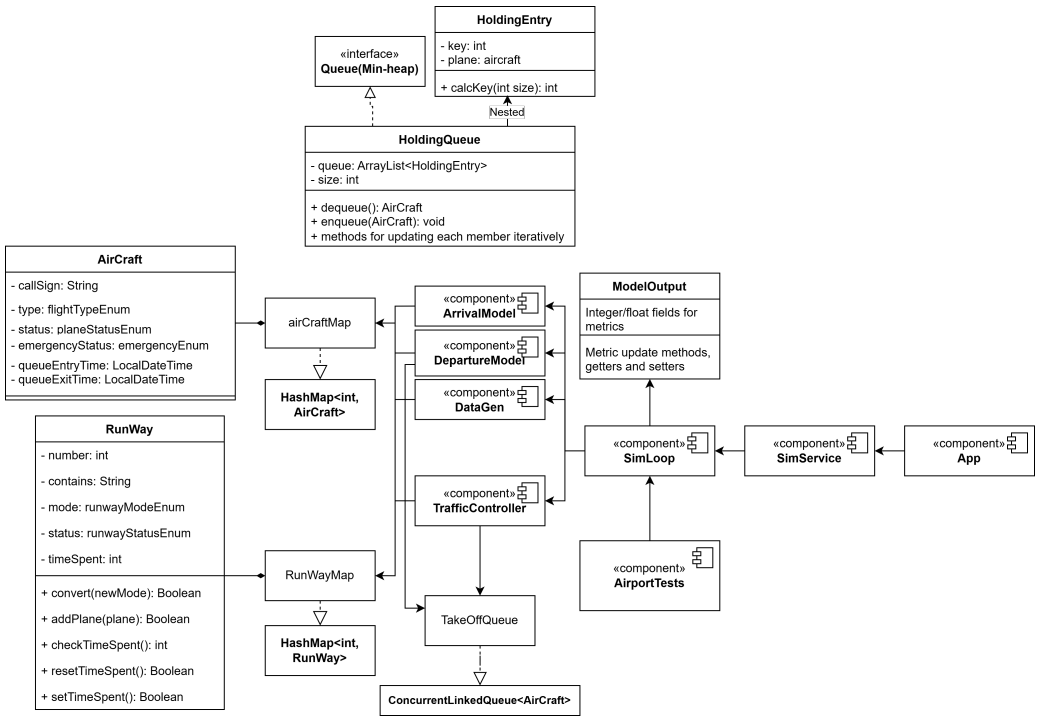


Figure 1: The original Class diagram for SimLoop.

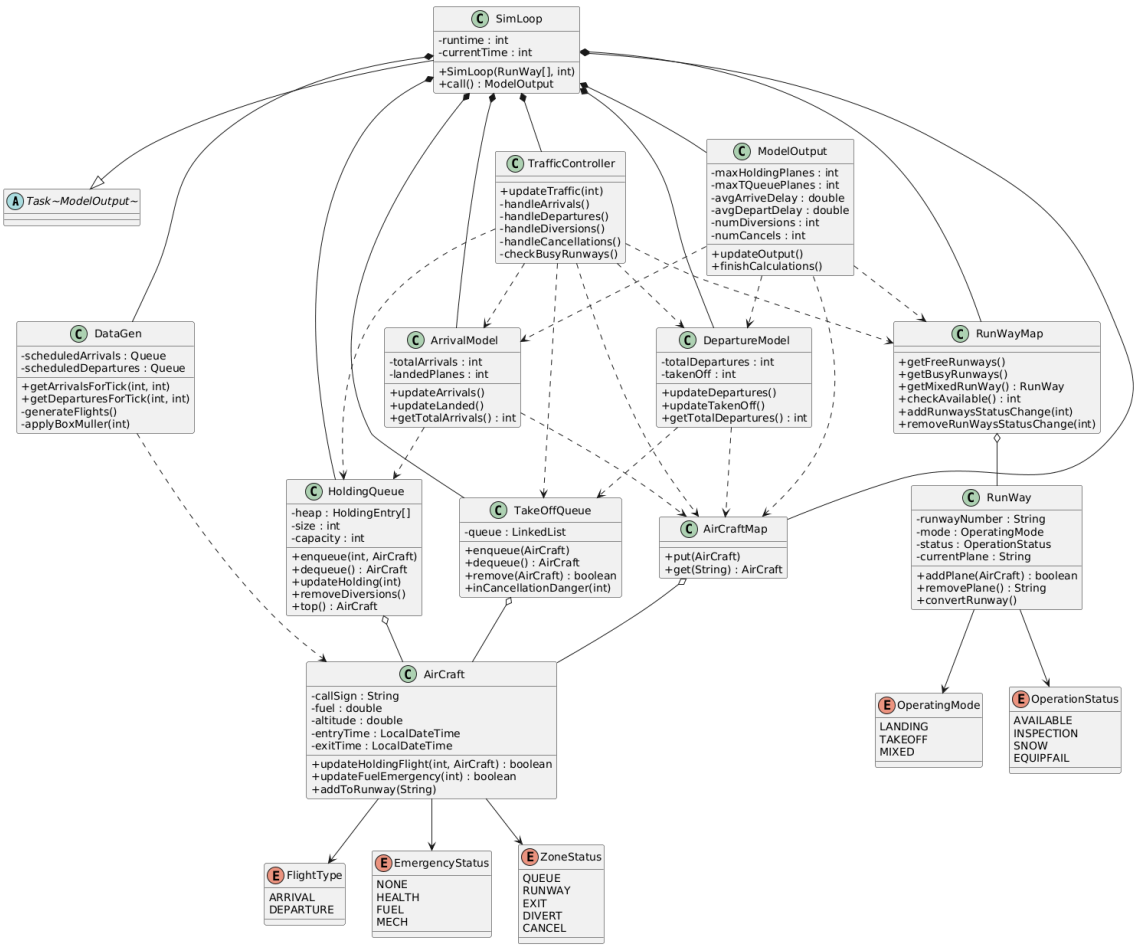


Figure 2: The actual Class diagram for SimLoop.

2 System Description

2.1 Core Features

The application aims to simulate departure traffic at a single airport over a configurable time period for arriving and departing aeroplanes. The application tracks data such as take-off queue size and average delay between scheduled and actual arrival times.

DataGen generates the entire flight schedule before the simulation starts and is the first code that is executed when a simulation starts. It uses two PriorityQueues holding `AirCraft` objects sorted by actual entry time, so each tick's flights can be retrieved efficiently. Flights are scheduled at a fixed interval, calculated using `arrivalInterval` $= 60 \div$ `arrivalRate` minutes. Since aircraft do not arrive exactly on schedule, we used the Box–Muller transformation, which converts two variables u_1, u_2 into a normally distributed offset with $\sigma = 5$ minutes. This produces aircraft which arrive both early and late. For departures, an additional uniform random offset of 10–30 minutes is added first (modelling gate clearance), then Box–Muller is applied on top.

The generated data is passed to `SimLoop.call()`, where one tick equals 1 minute of simulated time. After each loop of SimLoop: new aircraft whose actual entry time is in the interval [currentTime, currentTime+1] are spawned, queues are updated, TrafficController runs runway assignment logic and returns the callsigns of all planes that exited a queue this tick; ModelOutput records statistics to be output in the GUI. Exceptions within a tick are caught, and the loop continues using the continue function. This stops one error within a single tick from breaking the entire simulation.

The most technically advanced piece of backend code was our implementation of an array-backed `MinHeap`. The key reason we chose to implement this instead of using Java's `PriorityQueue` is that the Queue is treated as a black box and there is no way to change an item's priority once it has been inserted. Our implementation allows the can change a priority and then re-heapify. This is important for if a aeroplane reaches critical levels of fuel and must complete an emergency landing. Because of Java's inability to automatically re-heapify, the time complexity would be O(n), whereas our implementation is O(log n).

2.2 User Interface

When the User first opens the application, they see the start page where they can choose to either start a new simulation, see the user guide (holding key information on how the application works), or compare the key statistics of how previous simulations have gone.

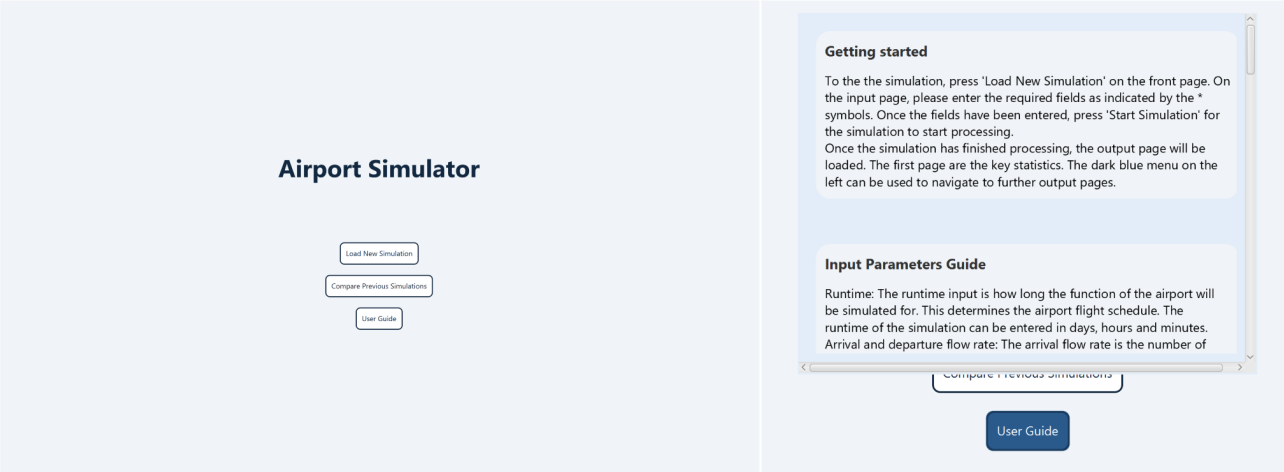


Figure 3: Start page and the User Guide.

The compare simulations page only displays the key statistics required by the client specification that is stored in a small JSON file locally on the user's computer. Storing full table or graph data for all saved simulations was not implemented since each simulation could contain thousands of tuples, consuming significantly more storage.

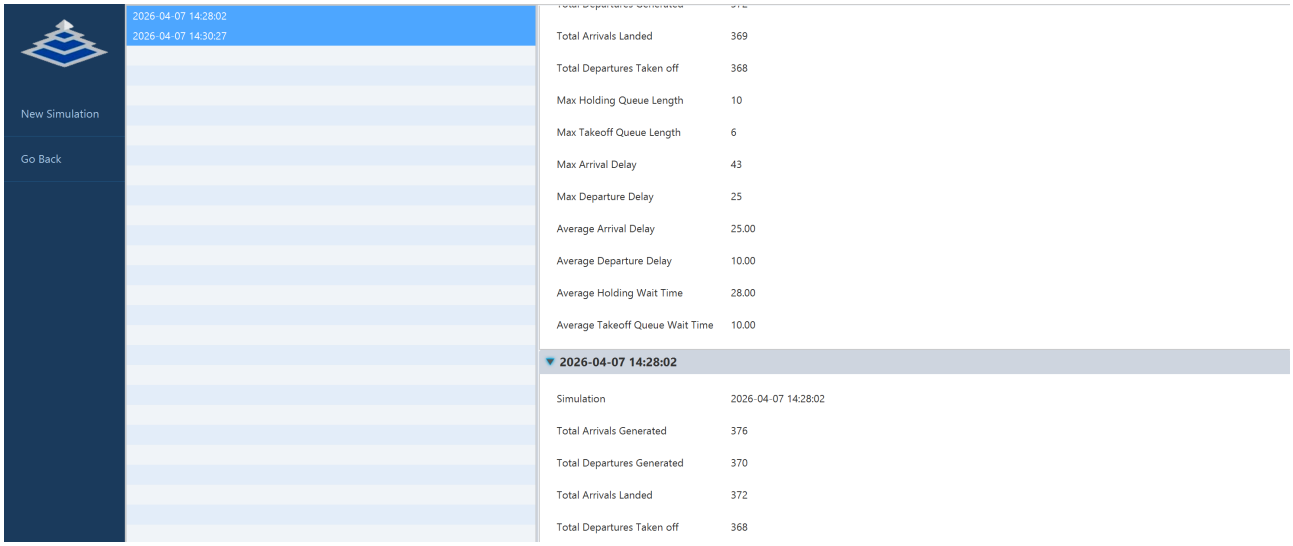


Figure 4: Compare page within the application

The simulation input page gives the user the ability to configure all input parameters, giving them as much freedom as possible to change the simulation to their scenario. The form is split up into two key sections: general simulation parameters and runway configuration, making it clear to the user what each group affects. Runways and status changes are dynamically loaded into the page using Java code, which makes the form less cluttered and only shows the user the inputs they need to see. When the user clicks the Start button, all the inputs are checked by a method called `CheckSim()`, which displays an alert to the user if they have made a mistake and then stops the simulation from starting.

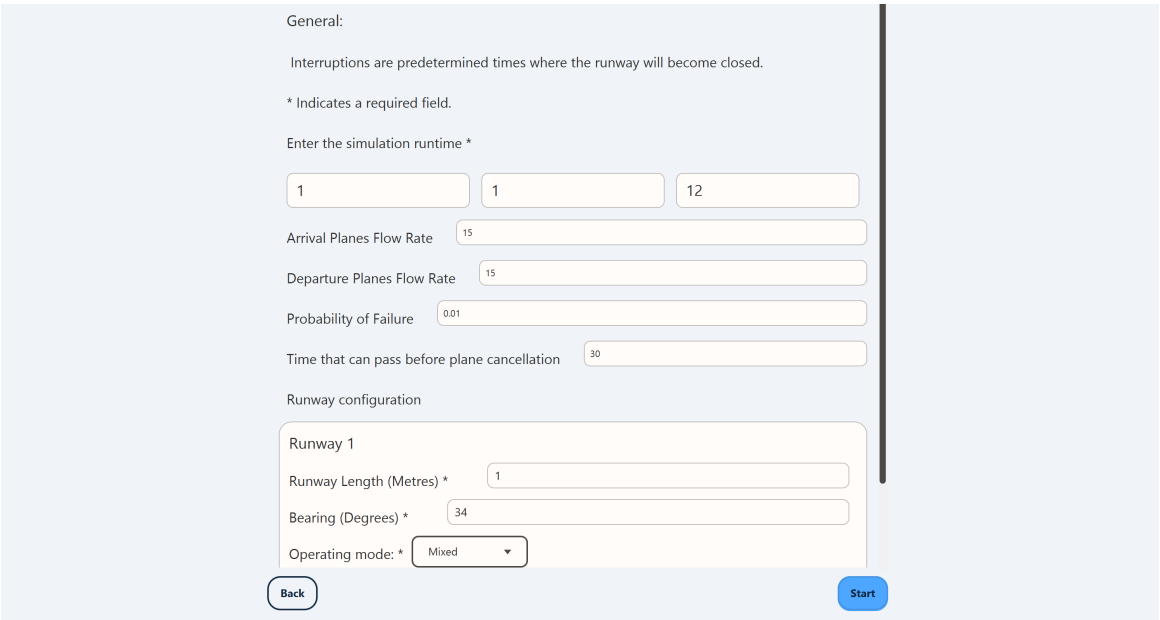


Figure 5: Simulation Input page

When the simulation completes, the user is taken to the key statistics page , which displays all values

specified by the client specification. The page is kept simple because development time was prioritised toward the more complicated parts of the results interface rather than the presentation of summary statistics.

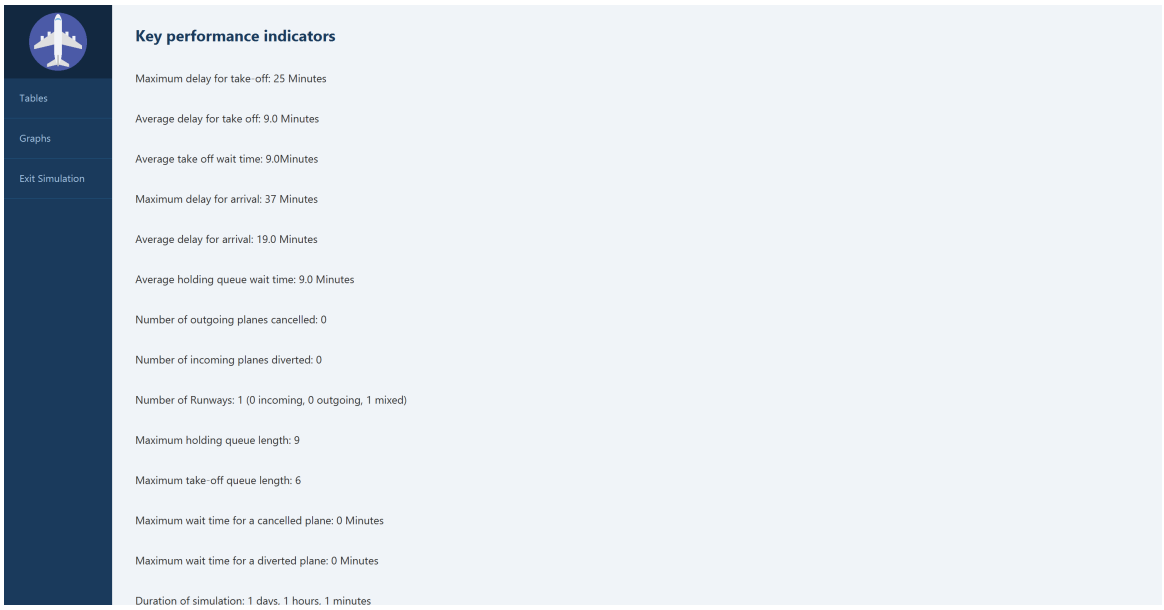


Figure 6: Simulation Key statistics page

The Key Statistics page allows the user to navigate to either the tables or graphs pages. The table pages offer five tables, each covering different aspects of the simulation at every timestamp. The user can filter the results between to timestamps of their choice, or cycle through 30-minute groups using the navigation buttons below the table. The reset button restores the dataset. The range queries used in the search is implemented using Java's `TreeMap` class with O(log(n+k)) time complexity, where n is the total number of entries and k is the number of results returned. The Runway table differs slightly and uses a `HashMap` of `TreeMap`s, where each `HashMap` entry holds all data for a specific runway, allowing efficient filtering by both runway and timestamp simultaneously. Below is a table of all data structures we considered and their time complexities:

Data Structure	**Time complexity**	**Reason why we did not choose it**
Array/ArrayList	O(N)	Linear search time is too slow for a simulation lasting multiple days as dataset could have thousands of entries.
HashMap	O(k) with discrete keys, O(N) generally,	A HashMap with discrete keys would offer O(1) per entry and so O(k) overall; it is less future proof. If the timestamps were changed to continuous values, the approach would break. The range query also requires a lookup per value in the range, which is costly for large intervals.
(Unbalanced) Binary Search Tree	O(N) (worst case)	Best case scenario is the same as a TreeMap, but no guarantee of the same result if the tree is not normalised.
Interval Tree	O(log(N+k))	Same time complexity as the TreeMap, but it is significantly harder to implement than a TreeMap because it has no inbuilt Java implementation, unlike TreeMap.
TreeMap	O(log(N+k))	This was the data structure we chose.

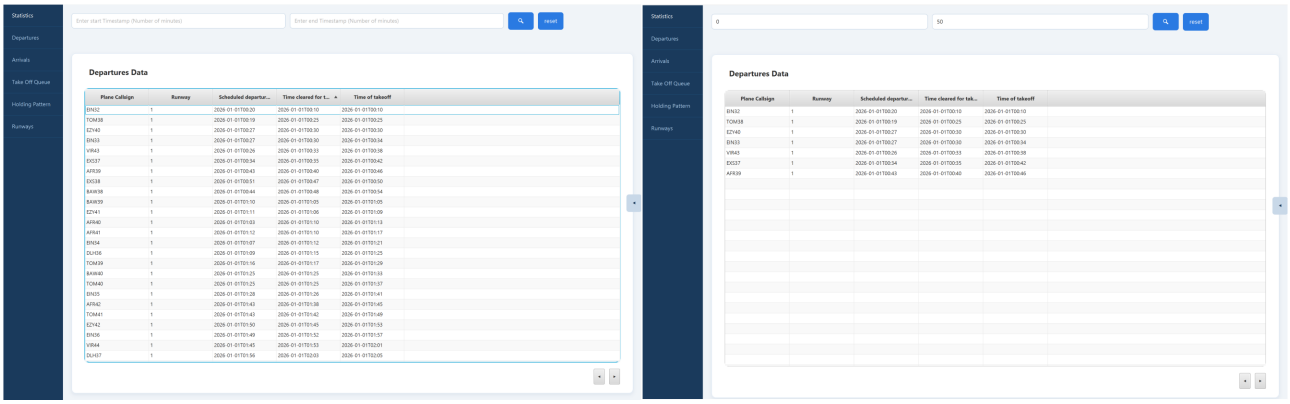


Figure 7: full table and filtered table for all results in the first 50 minutes

Since the planes are represented by their callsigns and runways their ID, a pop-up tab was implemented with information on each plane and runway, which can be filtered alphabetically by their callsign or runway number.Each row in the pop up table acts as a button, opening a tooltip wich detailed information about the aeroplane or runway.

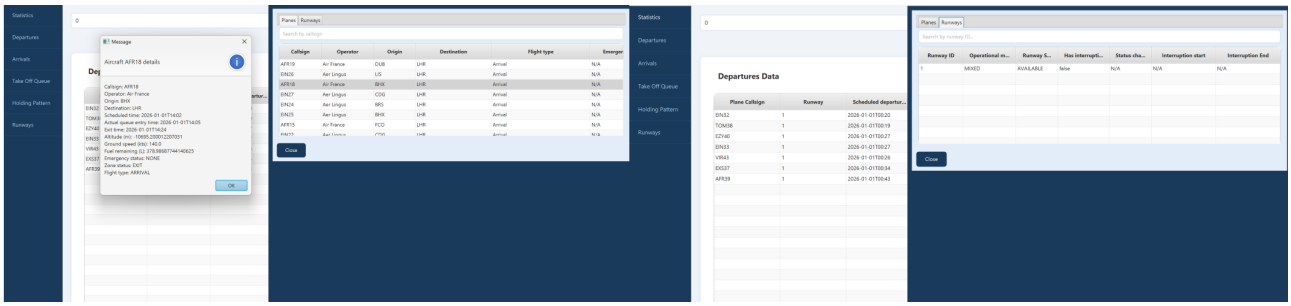


Figure 8: aeroplane pop-up table with tooltip and Runway pop-up table.

The Graphs page displays key relationships in the simulation over time. Since the graphs can sometimes be hard to read, we introduced tooltips that allow the user to see the specific data for that point. Graphs are generated by passing the coordinates to JavaFX's inbuilt `lineChart` object.

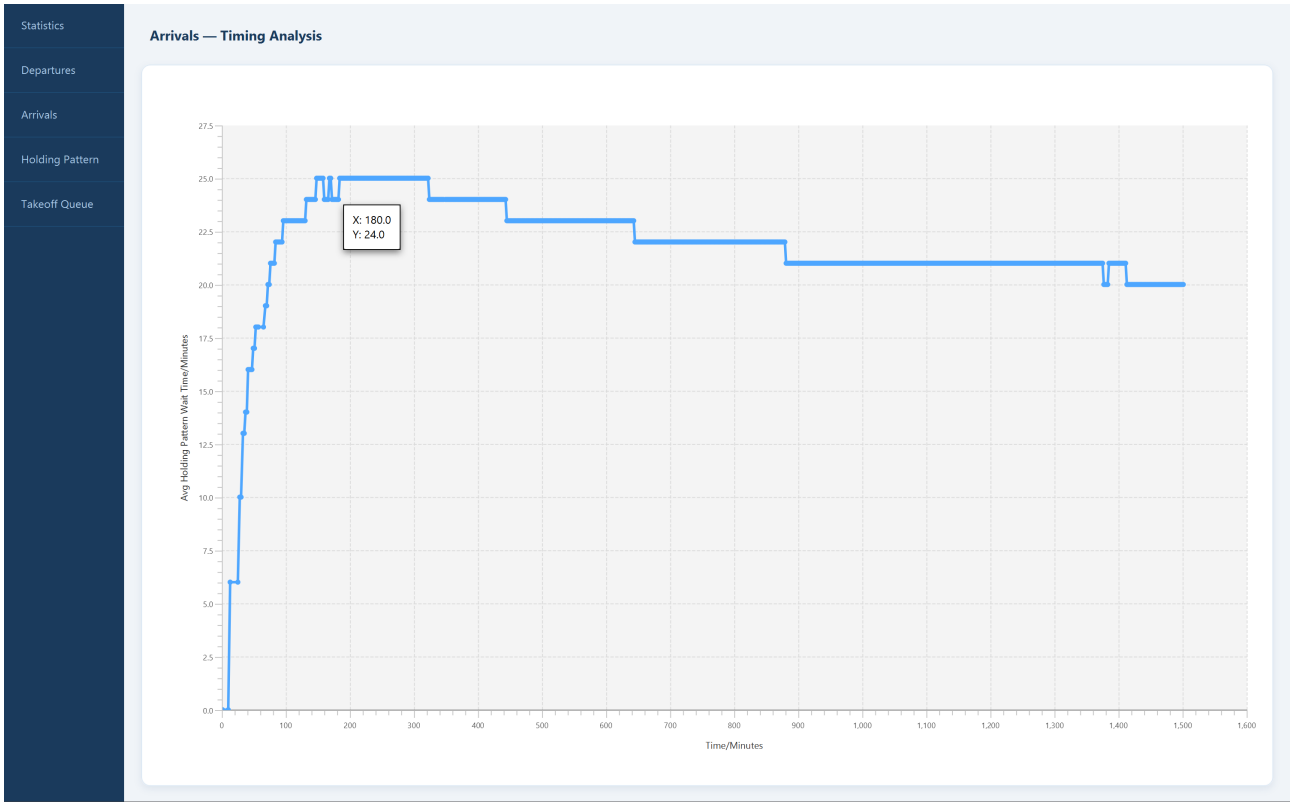


Figure 9: example graph with tooltip showing exact values of coordinate

The application consists of 15 key FXML pages, which can be navigated between via button presses within the application and in doing so, loads the FXML file into the primary stage. The following diagram shows a flow diagram for the application.

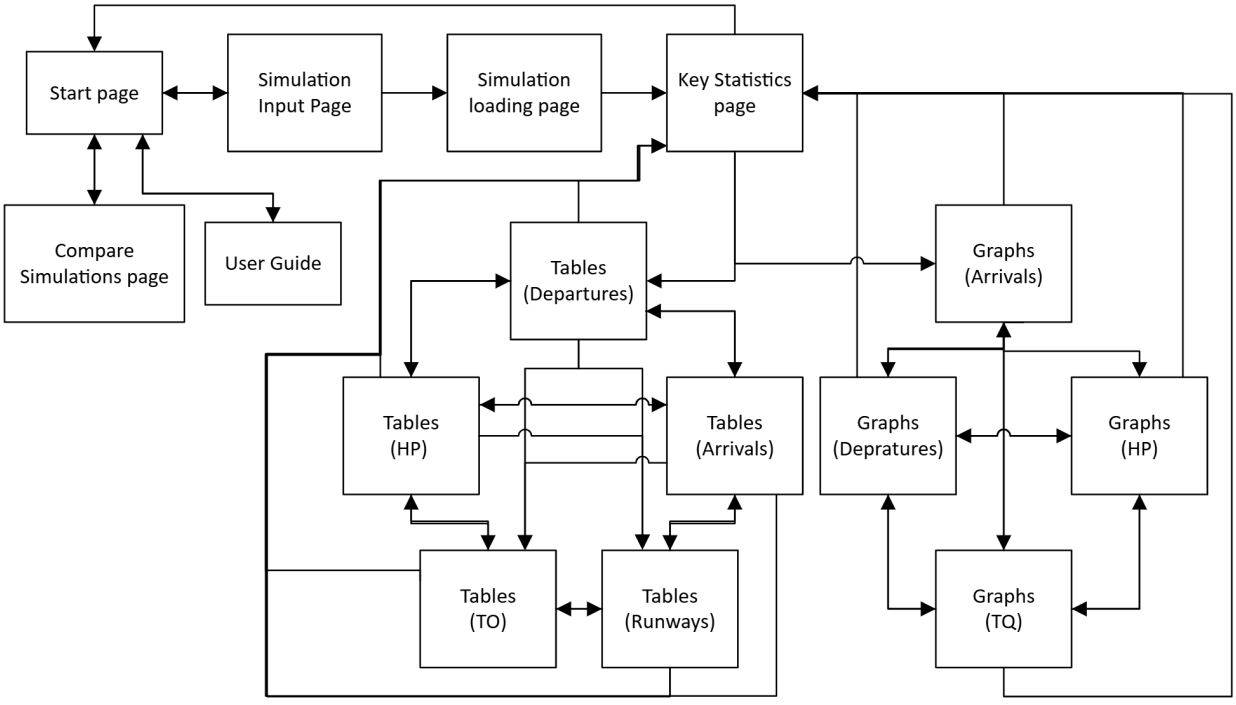


Figure 10: Flow diagram of the User interface.

Figure 10 shows the navigation flow through all screens in the application. The top row represents the core loop, while the diagram also highlights a clear structural divide in the post-simulation interface: the graphs and tables section, both accessible from the key statistics page, which highlights the split between the different formats of information.

3 System Evaluation

3.1 Requirements Coverage

All must-have requirements were either fully met or met with a justified modification. The most significant deviation was requirement 6.4, which was changed so that runway operational status changes were pre-scheduled rather than applied dynamically. This remained aligned with the client specification, which states "allows operators to specify a runway closure" [5], which our application does simply before the execution of the simulation. Some requirements were also changed in order to reflect the client's wishes after they spoke at the client consultation. For example, requirement 4.3 was changed since the clients stated they wanted all emergencies to be treated the same. Below is a Table covering the client requirements and their corresponding system features.

Table 4: Client Requirement coverage.

Client Requirement section	**Met?**	**System Feature**
Arrivals (Inbound)	Fully	Logic inside of `ArrivalModel.java`.
Departures (Outbound)	Fully	Logic inside of `DepartureModel.java`.
Runway Mode	Fully	Logic inside of `TrafficController.java`.
Input Parameters	Fully	All input parameters can be edited in the form used in `inputForm.fxml`.
Information	Fully	all tracked data is located in the data structures: `AirCraft.java`, `HoldingQueue.java`, `TakeOffQueue.java`, `RunWay.java`.
User Controls	Fully	Runway Operational Status and Runway Operating Mode can be changed in the form used in `inputForm.fxml`.
Output	Fully	All models data is present in `graphs(arrivals).fxml` and `graphs(departures).fxml` graphs pages, and everything else is in `keyData.fxml`.

The System also delivers several features beyond the original requirements. The tables, the holding pattern and take-off queue size graphs, the aircraft and the runway pop-up menu and the increased number of user-controlled input variables. These additions strengthen the systems' alignment with the client specification rather than distracting from it and serve the client's objective of modelling various airport scenarios and comparing the configurations.

The biggest unimplemented feature is a real-time simulation view, which was de-prioritised in favour of the features in the post-simulation results interface. While this did affect the application's modelling functionality, it does reduce the system's ability to show live simulation behaviour, which might be valuable to potential users. The addition of this feature would be of the highest priority in a future iteration.

Overall, the system is a strong prototype that satisfies all primary objectives set out by Dorset Software. Any requirements that were not fully met were developer-facing or UI features whose omission does not affect core functionality specified by the client.

3.2 Testing and Validation

Our testing philosophy was split up into 4 key sections: unit tests, integration tests, invariant tests and Ad Hoc tests. Each group was key in ensuring the system in its entirety functioned as expected [8].

Unit tests validate individual classes and their methods in isolation with no interaction between components. A helper method is called to create the dummy objects needed for the tests, which reduces the amount of repeated code that is present in the tests. For example, `UtilsTests` validated the tick-to-time conversion system, ensuring simulation timestamps are correct. Their benefits primarily include the ability to test foundational components with speed and precision without running the full simulation.

Integration tests verify specific input scenarios by running the entire simulation with fixed inputs and checking whether data generation, traffic controller decisions, queue management, and runway assignment all behave as expected. Since `SimLoop` extends JavaFX's `Task` class, it is designed to run in its own thread to prevent the freezing of the UI. An inner subclass called `TestableSimLoop` which calls `super.call()` and therefore bypasses the JavaFX threading system, allowing the simulation to run.

Invariant tests verify that properties of the simulation that must be true remain true. An example of one of these tests would be to check that all the output data in `ModelOutput` is non-null. This is essential because it allows us to know that the simulation's key properties will stay the same regardless of the input and in random generation in aircraft generation. The testing suite was made up of a total of 73 tests: 51 unit tests, 13 integration tests and 9 invariant tests, all of which passed.

```
[INFO] Tests run: 73, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.702 s
[INFO] Finished at: 2026-04-14T15:08:18+01:00
[INFO] ------------------------------------------------------------------------
```

Figure 11: Screenshot of testing interface.

Ad Hoc testing was conducted informally and was done by giving the uninvolved in the development of the UI and asking them to try to break it. If the testers managed to break the UI, then the problem was reported to the UI team and fixed accordingly, which uncovered errors that were not originally anticipated.

Because of the testing which was created by a separate member of the team, we were able to catch edge cases which members of the team may not have thought of before. If I were to improve the testing, I would have included a seed to the random generation used by `dataGen`. Without this, the same situation cannot be consistently regenerated to test the error. The tests are also missing certain edge cases, such as if all runways were closed or the outcome of extremely high outbound or inbound flow.

3.3 User Interface Evaluation

The User Interface was built with JavaFX, FXML layout files, CSS stylesheets, and Java controller logic. We chose JavaFX over Swing because of its native threading support, allowing the application and GUI to run in parallel. It also allowed us to use Scene Builder, which generates a live view of the FXML file. Because of its drag-and-drop behaviour, it does not require the developers to write FXML by hand, which is verbose and difficult to visualise. The Use of CSS-based styling allowed for consistent theming across screens without changing controller code.

In order to fully evaluate the User interface, I will be using Nielsen's Usability Heuristics[6]. Below is a table which states every Heuristic and how it was met.

Design Heuristic	**How it was met**
1: Visibility of System Status	A Loading bar displays how much of the simulation has been computed, reassuring the user that the application has not crashed during a long simulation runtime.
2: Match Between the System and the Real World	Almost all terminology is taken directly from the aviation industry, meaning the user is likely to understand the term "callsign" or "holding pattern" is. However, the use of the word "tick" is very software-oriented and may confuse non-technical users.
3: User Control and Freedom	The user can return to the main menu without saving to correct mistakes. If lost, the statistics button is accessible from all post-simulation pages and will take them back to the key data page, from which any other page in the application can be reached. However, the user cannot edit a simulation once it has run, so the user must recreate it from scratch.
4: Consistency and Standards	A consistent sidebar is present on all post-simulation pages. The input form also heavily resembles forms used on websites, making it intuitive to new users.
5: Error Prevention	`CheckSim()` validates user input to prevent incorrect or out-of-range values. Similar validation is also used for the search bars on the graphs pages.
6: Recognition Rather than Recall	Pop-up menu displays all required aircraft and runway data, meaning the user never needs to memorise information regarding aircraft and runways.
7: Flexibility and Efficiency of Use	Default values in the input screen allow novice users to start simulations immediately, while experienced users can change parameters to their situation. The compare simulations page compares previous outputs without needing to re-run the inputs. However, the lack of keyboard shortcuts would have greatly improved efficiency for experienced users.
8: Aesthetic and Minimalist Design	A colour palette of primarily blue is used throughout the entire application. Timestamp filtering allows users to focus on a smaller dataset.
9: Help Users Recognise, Diagnose, and Recover from Errors	Each error has a specific plain English message. For example, "Days must be between 0 and 20" for out-of-range numbers and "Invalid number of days for simulation length" for invalid input type.
10: Help and Documentation	A user guide that is visible from the start page and explains how the application works. However, this guide is only available from the start page, meaning the user has to leave their simulation in order to see it.

The biggest drawback of the UI backend is that the tables are stored in Java collections, rather than a database. Using a technology such as SQLite would have allowed for more detailed querying and, therefore, the future capability for extra features. An example is ordering flights by delay rather than only by exit time. The database would have managed memory usage more efficiently than the JVM heap and aligned with the principles of separation of concerns used in MVC architecture. The `TreeMap` approach was implemented since the only search in the requirements was by timestamp, and integrating a database within the allotted timeframe was not considered a priority.

Overall, the UI follows the majority of Nielsen's usability heuristics, especially numbers 5, 6 and 1. The largest violation of the Heuristics is the lack of keyboard shortcuts, and the user guide being inaccessible once the simulation has started. The decision to use Java collections rather than a database is the largest bottleneck for future innovations, although it was a necessary choice for the given development timeframe.

4 Team and Methodology

4.1 Methodology Choice

Our methodology did not change substantially from the one described in the design document. The team was split up into 4 key teams, each with its own responsibilities and parts of the project to work on. The teams were UI (Adrian and Adi), modelling (Afra and Prisha), testing (Yunas) and data generation (Rhys). We did this so that each team could focus on their own part of the project while reducing the risk of large teams working on the same pieces of code, such as merge conflicts.

While developing, we adopted a version of the SCRUM methodology, with one-week sprints and meetings twice a week. The Tuesday meeting was for sprint planning. Because of the lack of client consultation, we replaced the Scrum master with two representatives who would attend the biweekly seminars in order to gather extra client information. During the sprint review, each sub-team demonstrated its progress while the rest of the team evaluated it from the point of view of the client, quoting the specification.

Table 6: Summary of sprint 1 & 2 progress.

Sub-team	**Sprint 1 Task**	**Status**	**Sprint 2 Task**	**Status**
Modelling	Aircraft and Runway class	Done	TrafficController logic	Unfinished
Modelling	Holding Queue data structure	Done	ArrivalModel	Done
Modelling	Takeoff Queue data structure	Unfinished	DepartureModel	Done
Modelling	Airplane schedule generation	Done	ModelOutput	Done
UI	App navigation menu	Unfinished	App navigation menu	Done
UI	Simulation input page	Unfinished	Simulation input page	Done
Testing	Testing interface skeleton code	Done	Planning unit tests	Done

Table 7: Summary of sprint 3 & 4 progress.

Sub-team	**Sprint 3 Task**	**Status**	**Sprint 4 Task**	**Status**
Modelling	TrafficController logic	Done	ModelOutput	Done
Modelling	SimLoop (background task)	Unfinished	SimLoop	Done
Modelling	ModelOutput	Unfinished	—	—
UI	Statistics output page	Unfinished	Statistics output page	Done
UI	Tables page	Unfinished	Tables page	Done
UI	Graphs page	Unfinished	Graphs page	Done
UI	—	—	Airplane search	Done
UI	—	—	Table search	Done
Testing	Implement unit tests	Unfinished	Implement unit tests	Done

Sub-team	Sprint 3 Task	Status	Sprint 4 Task	Status
Testing	Implement invariant tests	Unfinished	Implement invariant tests	Done
Testing	—	—	Test using unit tests	Done
Testing	—	—	Test using invariant tests	Done
Extra features	Save simulation key statistics	Done	2D graphics	Discarded
Extra features	Load previous simulation statistics	Done	—	—

Table 8: Summary of sprint 5 progress.

Sub-team	Sprint 5 Task	Status
Testing	Re-test all tests after bug fixes	Done

The key tools were WhatsApp for quick questions, GitHub for version control, Google Docs for meeting notes and sprint checklists, and Teams for remote attendance. Each major feature was developed on a unique Git branch where daily pushes were encouraged, which resulted in fewer major conflicts. The majority of conflicts that did occur were caused by members not pulling whenever a large change occurred, which should have been enforced as standard practise. The Google doc was used to maintain and create meeting notes, seminar summaries and client consultation records and was used as a central store of information. However, the Google Doc had an inconsistent structure in some parts, which made finding certain resources difficult to locate. A good example of this is the sprint checklist, which was moved from the sprints page to the roles page, confusing some members.

The GitHub Kanban board that was specified in the design document was abandoned for a sprint checklist on Google Docs because the team found it unnecessary to learn how it worked, given the short development timeline. Each sub-team created a checklist of objectives for the upcoming sprint, with any Unfinished tasks carried over to the next sprint as a priority. Unlike a Kanban board, the checklist could not indicate which tasks were currently being completed. It also offered no mechanism for limiting the current workload across sub-teams.

Sprint 5

Section	Subtasks	Assigned to
Tests	☐ See if UI breaks	Yunas
Modelling	☐ Graphics	Afra
UI	☑ ~~Alert objects for errors - ie error popups~~ ☑ ~~Make input screen bigger & add some hints on what parts are required inputs etc, maybe a little bit of text that says what interrupts mean~~ ☐ Make KPI page look nicer ☐ Add 'tooltip' for stuff that might not be clear ie buttons ☑ ~~Departures table changes: actual arrival time becomes time cleared for take off, exit time becomes time of take off~~ ☑ ~~Arrivals table: actual arrival time change to airspace arrival time, exit time change to time of landing~~ ☑ ~~holding queue: actual arrival time & exit time should have same label as arrivals table~~ ☑ ~~take off queue tbl: actual time & exit same label as departure table~~ ☑ ~~scheduled and actual time columns need to be swapped before label changes~~ ☐ Button icons ☐ Accessibility features (low priority)	Adrian
Presentation	☐ Write presentation script	Rhys, Prisha

Figure 12: Screenshot of sprint checklist from the Google Doc.

The Key reason we decided to go with an adapted form of SCRUM instead of the other key alternative, waterfall, is that waterfall requires all requirements to be fixed and software is only delivered at the end of development. The key benefit of Scrum was that it allowed changes to be made dynamically [7], which was very important when the clients themselves made a change to requirement 4.3 halfway through development, which would not have been changed had waterfall been used. Although the team did not have daily standups because of a lack of time and other responsibilities, communication was still maintained through group chats on WhatsApp.

4.2 Methodology in Practice

Overall, the methodology was maintained well. Although not all planned tasks for every sprint were completed on time due to other responsibilities or other coworkers, team members focused on following the sprints, which then caused the correct amount of work to be completed. In the first few sprints, not many tasks were completed since the team underestimated how long it would take to learn the

new technologies required for the project, such as JavaFX for UI and JUnit5 for testing.

The majority of team members attended all meetings, allowing in-depth discussions about problems and design decisions. An example was the discussion of the format of `ModelOutput` so that the UI team knew how to process and display it correctly. The Tuesday meetings were very useful for the team to showcase to the rest of the team their progress and showcase completed work.

4.3 Risk Management

Throughout planning, the team identified several risks in the design document, including scope creep, technical issues during development and uneven workload distribution. An example of reducing scope creep includes the discarding of the real-time view and the Kanban board, which was discarded in favour of the checklist. Both of these are examples that show the team adjusting the scope because of the time pressure. The weekly meetings allowed these issues to be caught quickly. Technical issues also formed, such as the incomplete initial form of `ModelOutput`, which was documented in the "team coordination" risk. The use of GitHub branches and pushes allowed the team to work in parallel with minimal merge conflicts.

5 Individual Contribution

5.1 Overview of Contributions

As the leader of the Ui team, I was responsible for designing and implementing the majority of the User interface. Initially, I partitioned the work equally between myself and Adi. I frequently had to complete both his share and mine. Also, since Adi refused to learn how to use GitHub, I was left to manually merge his changes into my own branch, like the pop-up menu, which could not be merged automatically due to having over 20 merge conflicts. Because of this, I reduced the scope of the UI to what I believed I could deliver independently, and in the end had to delegate the use guide and compare simulations page to Afra, who had finished her modelling work.

5.2 Technical and Non-Technical Contributions

My most significant technical contribution was the design and implementation of the majority of the post-simulation results page, comprised of tables pages, graphs pages, key data pages and navigation system. I also developed the pre-simulation input page, which, in total, represents the main ways the user interacts with the application.

The most technical aspect was the timestamp-range query for the table pages. A two-week simulation with 1 aeroplane entering and leaving per minute would produce over 20,000 entries, which the UI needed to search without stalling. I implemented this using Java's inbuilt `TreeMap` class because its `subMap(fromKey, toKey)` method has a time complexity O(log(n+k)), which is efficient at scale.

The other technical challenge was the dynamic generation of JavaFX for the Runways and their status changes. With a maximum of 10 Runways and 3 status changes for each, a form showing all inputs would be overwhelming to the user. I implemented a `List<RunwayInputController>`, which was the controller for the `RunwayInput.fxml` page. This fxml object was dynamically loaded in using a `FXMLLoader` on a button press, with similar logic used for status changes. Delete buttons and input limits were introduced to allow for user error and input over-configuration.

My most significant non-technical challenge was coordinating with the modelling team to ensure that the format of `ModelOutput` was correct. Early in development, I discovered some data fields were in the wrong order, wrong format, or simply missing. An example was the `TreeMap` required its key to be the time in minutes, whereas `ModelOutput` showed the string that would be displayed in the table. This required me to communicate the missing requirements and the correct format for each, and therefore agree on how the data should be structured and passed to the UI. This Problem highlights the importance of creating detailed plans for component interfaces before beginning parallel development.

However, the Graphs page and the key statistics page could have been more polished. The Graph tooltip lacked CSS and axes labels, users could not zoom in on the graph, and the text in the key statistics page had no CSS or visual structure. As the team leader for UI, I should have defined the interface between the UI and modelling team more formally before beginning development.

5.3 Skills, Strengths, and Weaknesses

My most developed technical skill was a deeper understanding of JavaFX and Scene-Builder. Having no prior experience with JavaFX or UI design, I was able to design an intuitive UI that can present information clearly while preventing the user from being overwhelmed with information.

The most valuable non-technical skill I developed was communication during parallel team development. The `ModelOutput` issues described in Section 4.2 showed me that component interfaces are as important as the components themselves, and agreeing on data formats before development is essential. I also attended all team meetings and seminars, showcasing and explaining technical decisions and relaying seminar information to the group.

My biggest weakness during this project was planning and defining components and their interfaces before developing them in parallel. The time spent re-planning `ModelOutput` could have been spent developing new features and improving the functionality of the application. Ensuring there were no inconsistencies between the design and the requirement document would have meant all features would have been developed properly.

As UI team leader, my work bridged the simulation logic and the end user. Without the results interface, output of the modelling team could not be accessed, and without the validated input form, SimLoop had no data to run on. My attendance at the client consultation also ensured the change to requirement 4.3 was communicated and implemented correctly.

6 Conclusion

The requirements and design document captured the client's wants, but lacked some detail in sections. An example is measurable requirements with no SMART criteria and the lack of specification of the correct interfaces between components. The system delivered on all 7 sections of the specification, along with several additional features, making it genuinely useful for airport managers. The adapted SCRUM methodology was effective given the short development time frame and limited client access, although the lack of a kanban board reduced the correct spread of tasks between sprints.

References

[1] Kravchenko, T., Bogdanova, T., & Shevgunov, T. (2022). *Ranking Requirements Using MoSCoW Methodology in Practice*. In Computer Science On-line Conference (pp. 188–199). Springer.

[2] Hull, E., Jackson, K., & Dick, J. (1995). *SMART Requirements*. ACM SIGSOFT Software Engineering Notes, 20(2), pp. 42–47.

[3] Krasner, G. & Pope, S. (1988). *A Description of the Model-View-Controller User Interface Paradigm in the Smalltalk-80 System*. Journal of Object-Oriented Programming, 1(3), pp. 26–49.

[4] Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education Limited.

[5] Dorset Software Services Limited. (2025). *CS261 Project Specification 2026*. University of Warwick.

[6] Nielsen, J. & Molich, R. (1990). *Heuristic Evaluation of User Interfaces*. In Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '90), pp. 249–256. ACM Press.

[7] Beck, K. et al. (2001). *Manifesto for Agile Software Development*. Available at: https://agilemanifesto.org/

[8] Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.).

Declaration on the Use of Generative AI

This assignment has made use of Generative AI tools to support writing, formatting, and structural organisation. The final work reflects my own understanding and critical engagement with the material, and all use of such tools complies with the University of Warwick's guidelines on the responsible use of GenAI for assessed work. Specifically, the Large Language Model *Claude* (Anthropic) was used for the following purposes:

- **LaTeX Formatting and Structure:** To assist with the formatting of the LaTeX document, including the creation of tables, figures, and the BibTeX bibliography entries.
- **Proofreading and Tone Refinement:** To proofread drafted text for grammatical accuracy and refine sentence structures to ensure a formal academic tone appropriate for a software engineering report.
- **Structural Ideation:** To suggest how to structure and present critical analysis based on notes and project experiences provided by the author, helping to organise technical arguments into coherent academic prose.

All core concepts, project experiences, architectural decisions, technical analysis, and evaluations discussed within this report are entirely my own and accurately reflect my team's development of the Airport Modelling System.

Word Count: [5442] words (excluding diagram captions, citations and table headings).